ARCHITECTURE DESIGN DOCUMENT · v3

유압 설비
IoT-SCADA 아키텍처 설계서

v2 검토에서 나온 문제를 반영한 개정판. 기능 레이어(L1–L9)는 유지하고, 그 위에 배치 영역(OT · DMZ · IT)을 겹쳐 표시했다. 모든 구성요소를 도커 컨테이너로 배치하는 프로토타입을 전제로, 컨테이너 배치 뷰와 구현 순서까지 정리했다.

L9	프로세스 · 승인 · 앱	IT BPMN 엔진 · 운전원 가이드 앱
L8	AI 에이전트	IT LangGraph · MCP 도구 · LiteLLM
L7	온톨로지 · 원인분석	IT OWL 스키마 · Neo4j(n10s)
L6	관제 · SCADA · 인프라 감시	OT FUXA · IT Grafana · Prometheus · Alertmanager
L5	시계열 저장소	IT Kafka Connect(sink) · TimescaleDB
L4	스트림 처리 · CEP	IT Apache Flink (CEP + ONNX)
L3	이벤트 백본	DMZ Connect(ingest) · 명령 게이트웨이 · IT Kafka
L2	IoT 미들웨어 · MQTT	OT EMQX · EdgeX Foundry
L1	센서 네트워크 · 설비	OT 유압설비 · PLC · 고속 DAQ

구성: 1. v2 대비 변경 · 2. 설계 원칙 · 3. 아키텍처 구조도 · 4. 레이어 × 영역 구성 · 5. 데이터 흐름 · 6. 관제와 플랫폼 감시 · 7. 명령 경로와 안전 · 8. 지식·에이전트·HITL · 9. 시나리오 시퀀스 · 10. 도커 배치 뷰 · 11. 프로토타입 구현 순서

작성일: 2026-09-23

1. v2 대비 변경 요약

v2 검토 지적 16건 중 15건을 반영했다. 16번(FUXA 레이어 위치)은 기능 레이어 기준으로 L6가 맞아 레이어는 유지하고, 배치 영역만 OT로 표시했다.

#	v2 문제	v3 해결
1	센서가 PLC를 거치지 않아 인터록에 입력이 없음	센서 → PLC 입력으로 수정. EdgeX는 PLC 태그를 읽는다(device-modbus). 100 Hz 파형은 PLC가 아니라 **고속 DAQ**가 받아 EdgeX device-daq로 올린다.
2	SCADA 제어 경로 누락, 명령 출처 2곳 충돌	FUXA → `cmd/manual` 경로 추가. PLC에 **운전 모드**(LOCAL / REMOTE_MANUAL / REMOTE_AUTO)를 두어 모드와 출처가 일치하는 명령만 받는다. 모드 전환은 FUXA와 현장만 가능.
3	MQTT 단계에서 명령 만료 시각 소실, 중복 처리 미정	모든 구간 페이로드에 `cmdId · expiresAt` 유지. 게이트웨이와 PLC가 둘 다 만료를 검사하고, PLC가 최근 `cmdId` 32개로 중복을 걸러낸다. `cmd` 토픽은 retained 금지.
4	OT/IT 경계 없음, IT가 브로커 하나로 PLC까지 도달	OT · DMZ · IT 영역으로 분리. 두 영역을 잇는 것은 **DMZ 컨테이너 3개**(Connect ingest, 명령 게이트웨이, Prometheus Agent)뿐이다. 도커 네트워크도 `ot-net` / `it-net` 으로 나눈다.
5	100 Hz 배치 배열을 FUXA 태그·JDBC 행으로 못 받음	토픽을 `tag` (1 Hz 스칼라)와 `wave` (배치 배열)로 분리. FUXA는 `tag` 만 구독한다. TimescaleDB에는 1 Hz 태그와 1초 특징만 적재하고, 원시 파형은 Kafka에 7일 보관한다(선택: MinIO 아카이브).
6	가상 센서 CE·CP·SE 계산 위치 불명	PLC가 1 Hz로 계산해 태그로 노출한다(쿨러 입출구 온도·유량·모터 전력 기반). 값의 출처와 타임스탬프 기준이 PLC 하나로 정해진다.
7	다중 주기 시간축 정렬, 메시지 순서 미정의	이벤트 시간은 PLC·DAQ 타임스탬프를 쓰고, Flink 워터마크는 2 s, 허용 지연은 5 s. Connect ingest는 자산별 토픽 필터에 작업 1개(공유 구독 미사용)로 자산 내 순서를 보장한다.
8	Schema Registry(Avro)와 JSON 페이로드 불일치	프로토타입은 JSON + 문서화된 JSON Schema로 통일하고 Registry는 제외. 운영 전환 시 JSON Schema Registry로 도입.
9	경보 선이 Kafka를 우회해 그려짐	에이전트로 가는 경보 선을 Kafka `alerts` 에서 출발하도록 수정.
10	FUXA 알람 경로 미표시	명령 게이트웨이가 Kafka `alerts` 를 OT `plant/{asset}/alert` 로 중계하는 경로를 그림에 표시.
11	PLC 명령 ACK 경로 미표시	PLC → EdgeX → `status` → Connect ingest → Kafka `plant.status` → BPMN(ACK 대기)·audit 경로 추가.
12	시뮬레이터 루프 미완결, 실제 설비 구동 선 없음	"유압설비 / hyd-sim"을 한 박스로 합쳐 같은 인터페이스(센서 신호, 구동 입력)를 갖게 했다. 실제 설비와 시뮬레이터는 교체 가능한 구현이다.
13	MCP 도구 선 일부 누락	MCP → Neo4j · TimescaleDB · Prometheus 선을 모두 표시(읽기 전용).
14	Alertmanager 수신처 없음	Alertmanager → 인프라 담당 메일·웹훅(프로토타입은 Mailpit).
15	Kafka Connect 하나에 수집·적재·경보·명령 집중	Connect를 ingest(DMZ)와 sink(IT) 두 클러스터로 분리하고, 명령·경보 하향은 Connect 대신 전용 **명령 게이트웨이**가 맡는다.
16	FUXA 레이어 위치	철회. 기능 레이어는 L6 유지, 배치 영역은 OT로 표시. EMQX → FUXA는 "실시간 경로(Kafka·DB 우회)"로 명시.

2. 설계 원칙: 두 개의 축

v3는 구성요소를 두 기준으로 동시에 분류한다. **기능 레이어**는 "무슨 일을 하는가", **배치 영역**은 "어느 네트워크에서 실행되는가"를 말한다. 레이어 번호는 데이터가 가공되는 단계를 뜻하고, 영역은 보안 경계를 뜻한다. 같은 레이어 안에서도 영역이 다를 수 있다(예: L6의 FUXA는 OT, Grafana는 IT).

영역	원칙	도커 네트워크
OT	설비와 실시간 운전에 필요한 것만 둔다. IT가 멈춰도 감시(FUXA)와 수동 조작, 인터록은 계속 동작해야 한다.	`ot-net`
DMZ	OT와 IT를 잇는 유일한 통로. 올라가는 데이터는 복제만 하고, 내려가는 것은 검증된 명령과 알람 두 종류뿐이다.	`ot-net` + `it-net` 동시 연결
IT	장부·저장·분석·지식·에이전트·프로세스. 설비에 직접 연결하지 않는다.	`it-net`

v2에서 유지하는 원칙	내용
레이어당 핵심 기능 1개	같은 역할에 기술 하나만 둔다.
Kafka를 IT의 단일 장부로	IT 쪽 모든 소비자는 Kafka에서 읽고, 승인된 명령도 `action.cmd`에 기록된다.
설비 경보와 플랫폼 경보 분리	설비 이상은 Flink → 에이전트·운전원, 플랫폼 이상은 Prometheus → 인프라 담당.
지식은 온톨로지에	원인·조치·SOP·매뉴얼 관계는 Neo4j 그래프로 모델링하고, 에이전트는 노드 ID를 인용한다.
HITL과 다중 보호	사람 승인 → 게이트웨이 검증 → PLC 모드·만료·중복·인터록 검사를 모두 통과해야 실행된다.

3. 아키텍처 구조도

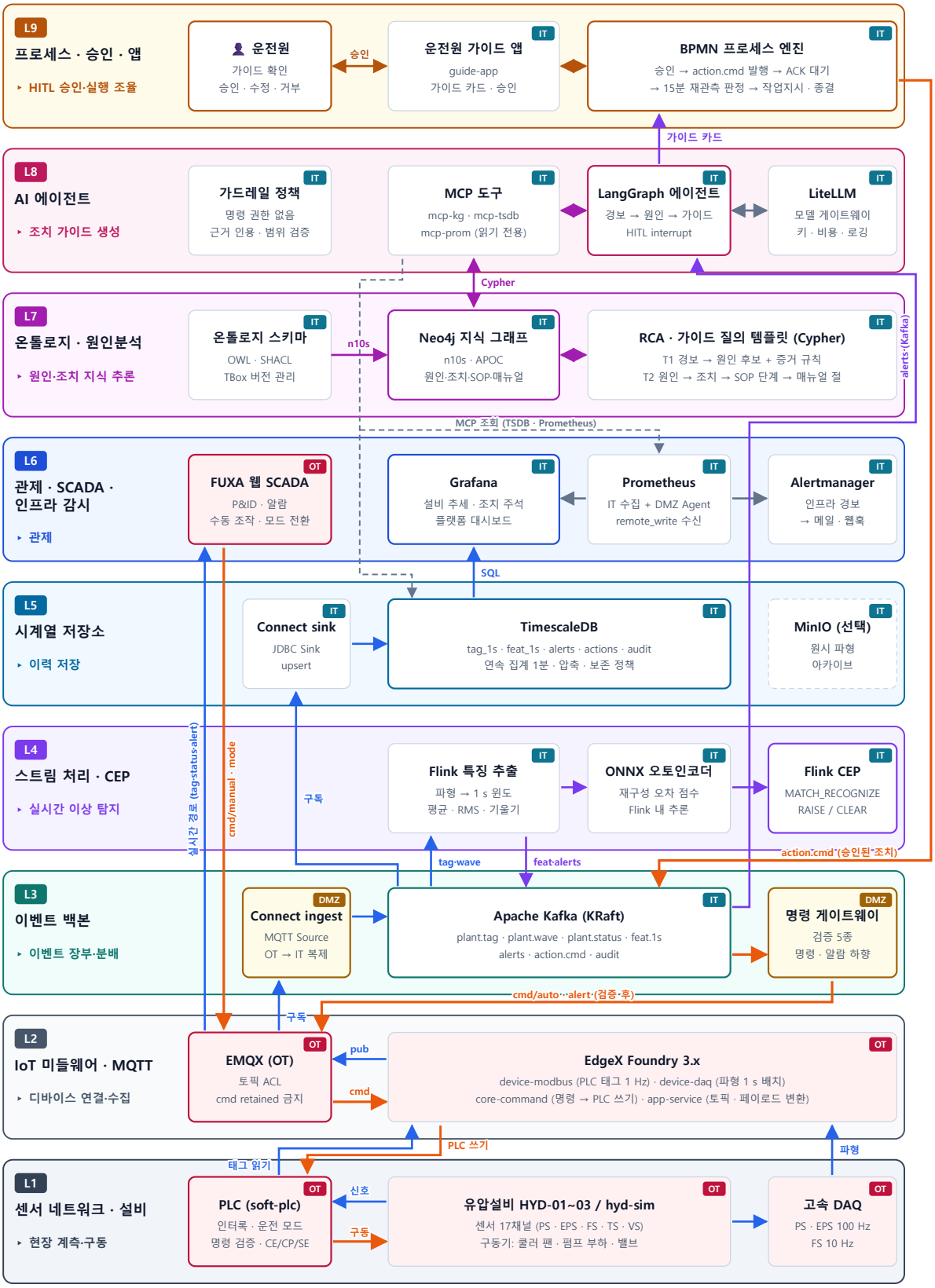


그림 1. v3 아키텍처 — 박스 배지와 바탕색은 배치 영역(OT DMZ IT). 선: — 데이터(상향) · — 경보·분석 · — 명령(하향) · — 승인 · --- 조회

4. 레이어 × 영역 구성

L	핵심 기능	구성요소 (컨테이너)	영역	역할
L1	**현장 계측·구동**	유압설비 HYD-01~03 / `hyd-sim`	OT	센서 17채널과 구동기(팬·부하·밸브). 시뮬레이터는 같은 신호 인터페이스를 제공하고 조치에 물리적으로 반응한다.
		PLC / `soft-plc`	OT	센서 입력, 인터록, 운전 모드, 명령 검증(만료·중복), CE·CP·SE 계산.
		고속 DAQ (`hyd-sim` 내장)	OT	PS·EPS 100 Hz, FS 10 Hz 파형 수집.
L2	**디바이스 연결·수집**	EdgeX Foundry 3.x (`device-modbus` , `device-daq` , `core-command` , `app-service`)	OT	PLC 태그(1 Hz)와 DAQ 파형(1초 배치)을 읽어 표준 토픽·페이로드로 발행한다. 하향 명령은 core-command가 PLC 쓰기로 바꾼다.
		EMQX (`emqx-ot`)	OT	OT 내부 MQTT 브로커. FUXA·EdgeX·DMZ 구독자에 분배하고 토픽 ACL로 발행 권한을 강제한다.
L3	**이벤트 장부·분배**	Kafka Connect ingest (`connect-ingest`)	DMZ	OT MQTT → Kafka 단방향 복제(MQTT Source).
		Apache Kafka KRaft (`kafka`)	IT	IT의 단일 이벤트 장부. 재처리 가능.
		명령 게이트웨이 (`cmd-gateway`)	DMZ	Kafka `action.cmd` · `alerts` 를 검증해 OT `cmd/auto` · `alert` 로 내려보내는 유일한 하향 통로.
L4	**실시간 이상 탐지**	Apache Flink (`flink-jm` , `flink-tm`) + PyFlink 잡	IT	파형 → 1초 특징 → ONNX 오토인코더 점수 → CEP 패턴 → `alerts` RAISE/CLEAR.
L5	**이력 저장**	Kafka Connect sink (`connect-sink`)	IT	Kafka → TimescaleDB 적재(JDBC Sink, upsert).
		TimescaleDB (`timescaledb`)	IT	1 Hz 태그, 1초 특징, 경보, 조치, 감사.
L6	**관제**	FUXA (`fuxa`)	OT	운전 화면·설비 알람·수동 조작·모드 전환. IT 장애와 무관하게 동작.
		Grafana (`grafana`)	IT	설비 추세·조치 주석, 플랫폼 대시보드.
		Prometheus (`prometheus`) + Agent (`prom-agent` , DMZ)	IT	플랫폼 메트릭. OT 쪽은 DMZ의 Agent가 수집해 remote_write로 보낸다.
		Alertmanager (`alertmanager`) → `mailpit`	IT	인프라 경보 라우팅·억제.
L7	**원인·조치 지식 추론**	Neo4j 5 + n10s + APOC (`neo4j`)	IT	온톨로지 스키마·인스턴스, RCA·가이드 Cypher 템플릿, 매뉴얼 청크 벡터 인덱스.
L8	**조치 가이드 생성**	`agent` (LangGraph), `mcp-kg` · `mcp-tsdb` · `mcp-prom` , `litellm`	IT	경보 → 데이터 신뢰 확인 → 원인 후보·증거 평가 → 가이드 카드. 명령 발행 권한 없음.
L9	**HITL 승인·실행 조율**	`bpmn` (Process GPT 또는 Flowable), `guide-app`	IT	승인 → `action.cmd` 발행 → ACK 대기 → 15분 재관측 → 작업지시·종결.

5. 데이터 흐름

5.1 OT MQTT 토픽 (`emqx-ot`)

토픽	방향	발행	구독	페이로드 · 규칙
`plant/{a}/tag/{name}`	↑	EdgeX	FUXA, connect-ingest	1 Hz 스칼라 `{t, v, q}`. FUXA 태그 바인딩 단위
`plant/{a}/wave/{sensor}`	↑	EdgeX	connect-ingest만	1초 배치 `{t0, hz, v[]}`. FUXA 구독 금지(ACL)
`plant/{a}/status`	↑	EdgeX	FUXA, connect-ingest, cmd-gateway	운전 모드, 명령 ACK `{cmdId, result}`. retained 허용
`plant/{a}/mode`	↓	FUXA	EdgeX	모드 전환 요청. FUXA 계정만 발행
`plant/{a}/cmd/manual`	↓	FUXA	EdgeX	수동 조작. `{cmdId, source:"FUXA", expiresAt, writes[]}`
`plant/{a}/cmd/auto`	↓	cmd-gateway	EdgeX	승인된 자동 조치. 형식 동일, `source:"HITL"`. retained 금지
`plant/{a}/alert`	↓	cmd-gateway	FUXA	IT 경보 중계(표시 전용)

5.2 Kafka 토픽 (`kafka`)

토픽	생산자 → 소비자	키	보존
`plant.tag`	connect-ingest → Flink, connect-sink	asset	7일
`plant.wave`	connect-ingest → Flink (선택: S3 Sink → MinIO)	asset	7일
`plant.status`	connect-ingest → BPMN(ACK 대기), connect-sink(audit)	asset	30일
`feat.1s`	Flink → connect-sink, 에이전트	asset	30일
`alerts`	Flink → 에이전트, BPMN, cmd-gateway(OT 중계), connect-sink	asset	1년
`action.cmd`	BPMN → cmd-gateway, connect-sink(actions)	asset	1년
`audit`	BPMN · cmd-gateway(거부 사유 포함) → connect-sink	incident	5년

5.3 TimescaleDB 테이블

테이블	내용	정책
`tag_1s`	1 Hz 태그(TS1–4, VS1, CE, CP, SE, 모드 등)	hypertable, 1분 연속 집계, 90일 → 집계 2년
`feat_1s`	파형 특징(평균·RMS·기울기)과 AE 점수	hypertable, 1분 연속 집계
`alerts` · `actions` · `audit`	경보 상태, 조치와 ACK, 승인·거부 감사	id 기준 upsert (RAISE → CLEAR 갱신)

> **원시 파형을 DB에 넣지 않는 이유.** 100 Hz 배열을 행으로 펼치면 설비 3기만으로 초당 약 2,200행이 된다. 탐지는 Flink가 Kafka에서 직접 하므로 DB에는 특징만 있으면 되고, 재분석이 필요하면 Kafka 7일 보관분이나 MinIO 아카이브를 다시 읽는다. 이렇게 하면 JDBC Sink가 배열을 다룰 필요가 없다(#5).

5.4 시간과 순서

- **이벤트 시간**은 PLC(태그)와 DAQ(파형 `t0`)가 찍는다. 컨테이너는 호스트 시계를 공유하므로 프로토타입에서 별도 동기화는 필요 없고, 실제 설비에서는 NTP/PTP로 맞춘다.
- Flink 워터마크 2 s, 허용 지연 5 s. 늦은 데이터는 side output으로 분리해 기록한다.
- 자산 내 순서: connect-ingest는 자산별 토픽 필터에 작업 1개를 두고(공유 구독 미사용), Kafka 키를 asset으로 해서 파티션 내 순서를 유지한다.

6. 관제와 플랫폼 감시

도구	영역	데이터 소스	역할
FUXA	OT	emqx-ot: `tag` · `status` · `alert`	P&ID·실시간 값·알람, 수동 조작과 모드 전환. IT가 끊겨도 동작하며, 이때는 경보 중계만 멈춘다.
Grafana	IT	TimescaleDB, Prometheus	설비 추세(`tag_1s` · `feat_1s`), 경보·조치 주석, 플랫폼 대시보드.
Prometheus	IT	IT exporter 직접 수집 + `prom-agent` (DMZ) remote_write	Kafka 랙, Flink 체크포인트, DB 지연, EMQX·EdgeX·PLC 연결 상태, `cmd-gateway` 거부 건수.
Alertmanager	IT	Prometheus 규칙	인프라 담당 메일·웹훅(프로토타입 Mailpit). 설비 경보는 보내지 않는다.

IT의 Prometheus가 OT로 직접 스크레이프하면 IT → OT 연결이 생긴다. 그래서 DMZ의 Prometheus Agent가 OT를 수집해 IT로 밀어 보내는 방식(OT → DMZ → IT)을 택했다.

7. 명령 경로와 안전

7.1 운전 모드와 명령 소유권 (PLC)

모드	받는 명령	설명	전환 권한
LOCAL	현장 패널만	정비·시운전. 원격 명령은 전부 거부	현장
REMOTE_MANUAL	`cmd/manual` (FUXA)	기본 운전 모드. HITL 조치 명령은 거부	현장, FUXA
REMOTE_AUTO	`cmd/auto` + `cmd/manual`	HITL 조치 허용. 수동 명령이 우선하고, 수동 조작이 들어오면 REMOTE_MANUAL로 자동 복귀	현장, FUXA

두 출처가 동시에 쓰는 문제(#2)는 이 규칙으로 정리된다. 사람(FUXA)이 언제나 자동 조치보다 우선하고, 자동 조치를 받을지 말지도 사람이 모드로 결정한다.

7.2 구간별 검증

구간	검증 항목	실패 시
L9 BPMN	사람 승인, 파라미터가 온톨로지 `paramRange` 안인지, `expiresAt` = 승인 + 120 s 부여	발행 안 함
DMZ cmd-gateway	① 스키마 ② 액션 화이트리스트 ③ 만료 ④ `cmdId` 중복 ⑤ `status`의 현재 모드가 REMOTE_AUTO인지, 초당 명령 수 제한	폐기 + audit에 거부 사유
OT EdgeX	토픽별 발행 계정 확인(ACL), 페이로드를 PLC 쓰기로 매핑. `source`·`cmdId`·`expiresAt`을 PLC로 그대로 전달	무시
OT PLC	모드와 출처 일치, 만료(PLC 시계), 최근 `cmdId` 32개 중복, 쓰기 범위, 하드 인터록(TS1 > 65 ℃ 트립, 최소 부하 60 %, 팬 과전류)	`result:"REJECTED"` + 사유 코드

7.3 메시지 흐름 (승인된 조치)

```
// Kafka action.cmd  (BPMN → cmd-gateway)
{ "cmdId":"CMD-0923-0042", "asset":"HYD-01",
  "incident":"INC-0923-01", "source":"HITL",
  "actions":[{"code":"FAN_BOOST","fan_pct":100},
             {"code":"REDUCE_LOAD","load_pct":80}],
  "approvedBy":"OP-17",
  "expiresAt":"2026-09-23T10:16:05Z" }
```

```
// OT MQTT plant/hyd01/cmd/auto  (cmd-gateway)
{ "cmdId":"CMD-0923-0042", "source":"HITL",
  "expiresAt":"2026-09-23T10:16:05Z",
  "writes":[{"res":"FanSpeedSP","v":100},
            {"res":"LoadSP","v":80}] }
// ACK plant/hyd01/status  (EdgeX ← PLC)
{ "mode":"REMOTE_AUTO", "cmdId":"CMD-0923-0042",
  "result":"DONE", "interlock":"PASS" }
```

ACK는 `status` → connect-ingest → Kafka `plant.status`로 올라가고, BPMN이 `cmdId`로 대조한다. 30초 안에 ACK가 없거나 REJECTED면 BPMN이 운전원에게 실패를 알리고 재시도하지 않는다.

8. 지식 · 에이전트 · HITL (L7–L9)

온톨로지 스키마는 v2 설계를 유지한다. 증상 → 고장모드 → 원인 → 증거로 원인을 추론하고, 원인 → 조치 → SOP 단계 → 매뉴얼 절로 조치를 안내한다. 연쇄 관계(LEADS_TO)로 결과를 원인으로 오인하지 않게 한다.

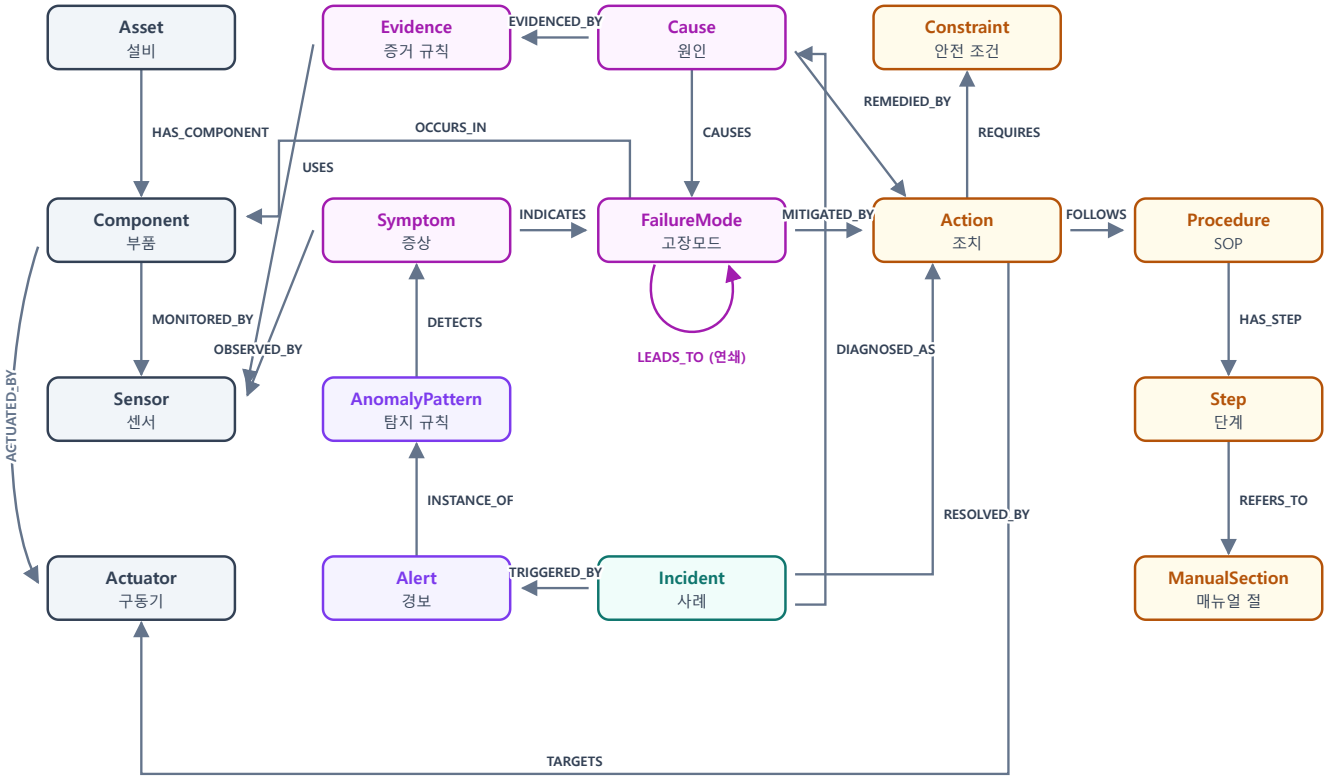


그림 2. 온톨로지 스키마(TBox) — ■ 자산 · ■ 관측 · ■ 고장 지식 · ■ 조치 지식 · ■ 사례

MCP 도구 (컨테이너)	기능	권한
`mcp-prom`	데이터 신선도·컨슈머 랙·ingest 상태 확인. 이상이면 추론을 보류하고 "데이터 신뢰 불가" 카드를 낸다	읽기
`mcp-kg`	Neo4j 화이트리스트 템플릿(T1 원인 후보, T2 조치 경로)과 매뉴얼 벡터 검색	읽기
`mcp-tsdb`	Evidence 노드의 SQL 템플릿 실행(`tag_1s` , `feat_1s`)	읽기
`bpmn` API	가이드 카드 제출(에이전트의 유일한 쓰기)	쓰기

> 에이전트 컨테이너는 `it-net` 에만 붙고 Kafka에는 `alerts` · `feat.1s` 읽기 권한만 가진다. `action.cmd` 쓰기 권한은 BPMN 서비스 계정에만 있다. 따라서 에이전트는 구조적으로 설비에 명령을 낼 수 없다.

9. 시나리오 시퀀스: 쿨러 성능 저하 → 과열 → HITL 조치

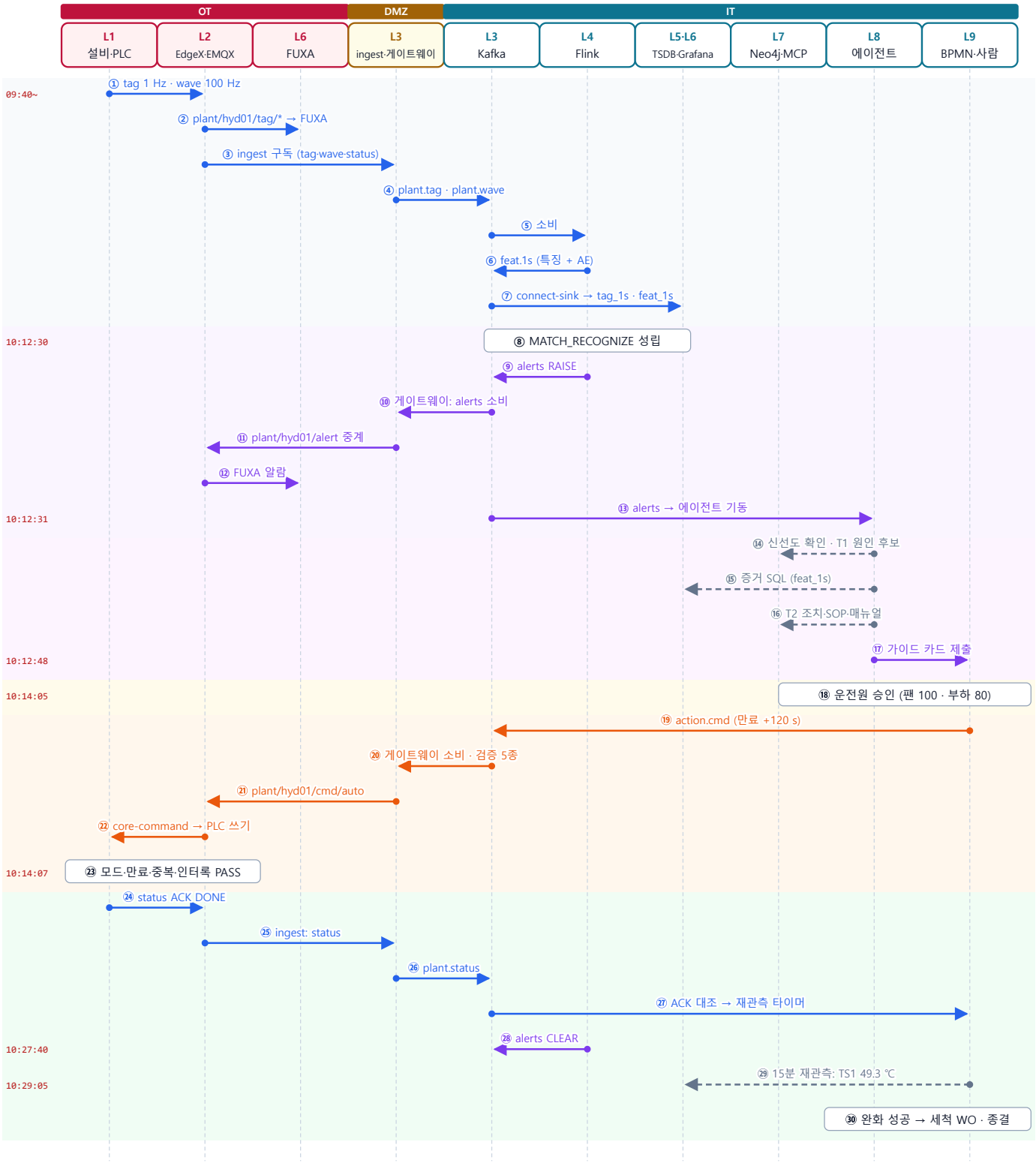


그림 3. v3 경로 기준 시퀀스 — 레인 머리 배지는 배치 영역. 명령은 DMZ 게이트웨이를 거쳐서만 OT로 들어간다

10. 도커 배치 뷰

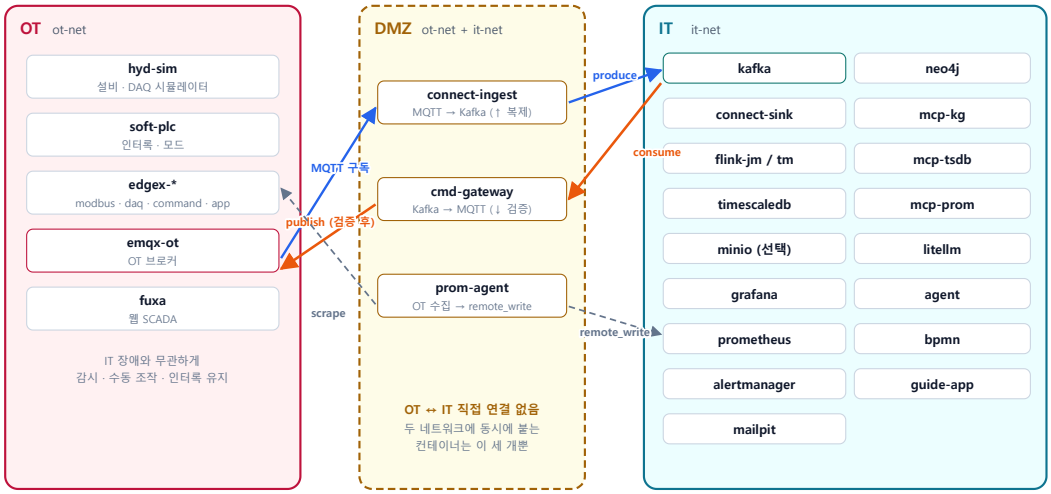


그림 4. 도커 네트워크와 컨테이너 — `ot-net`과 `it-net`에 동시에 붙는 컨테이너는 DMZ의 세 개뿐이다

서비스	이미지 (예)	네트워크	포트	비고
hyd-sim	자체 (python:3.12-slim, pymodbus)	ot	5020 (Modbus), 5021 (DAQ)	물리 모델·결함 주입·시간 가속
soft-plc	OpenPLC Runtime 또는 자체 Python	ot	502	인터록·모드·CE/CP/SE·명령 검증
edgex-*	edgexfoundry/core-keeper · core-metadata · core-command · device-modbus · app-service-configurable, 자체 device-daq	ot	59880–59900	메시지 버스로 emqx-ot 사용(`edgex/#` 네임스페이스)
emqx-ot	emqx/emqx:5	ot	1883, 18083	ACL 파일, retained 정책
fuxa	frangoteam/fuxa	ot	1881	프로젝트 파일 볼륨
connect-ingest	Kafka Connect + MQTT Source 커넥터	ot + it	8083	자산별 작업 1개
cmd-gateway	자체 (Python, aiokafka, paho-mqtt)	ot + it	8090 (/metrics)	검증 5종, 알람 중계
prom-agent	prom/prometheus (agent 모드)	ot + it	—	OT 수집 → remote_write
kafka	apache/kafka (KRaft)	it	9092	프로토타입 1노드, 운영 3노드
connect-sink	Kafka Connect + JDBC Sink	it	8084	upsert 모드 · (선택) S3 Sink → minio
flink-jm / flink-tm	flink:1.20 기반 자체 (PyFlink, onnxruntime)	it	8081	체크포인트 볼륨
timescaledb	timescale/timescaledb (pg16)	it	5432	데이터 볼륨, init SQL
grafana · prometheus · alertmanager · mailpit	grafana/grafana, prom/prometheus, prom/alertmanager, axllent/mailpit	it	3000, 9090, 9093, 8025	프로비저닝 파일로 대시보드·규칙 코드화
neo4j	neo4j:5 (플러그인 apoc, n10s)	it	7474, 7687	스키마·인스턴스 적재 스크립트
mcp-kg · mcp-tsdb · mcp-prom	자체 (Python FastMCP)	it	내부 전용	읽기 전용 계정
litellm · agent	ghcr.io/berriai/litellm, 자체 (LangGraph)	it	4000, —	API 키는 .env
bpmn · guide-app	Process GPT 또는 flowable/flowable-rest, 자체 웹앱	it	8080, 3001	승인 UI

11. 프로토타입 구현 순서 (바이브코딩용)

한 번에 전부 올리지 않고, 각 단계가 끝날 때마다 눈으로 확인할 수 있는 결과를 기준으로 진행한다. 도커 Compose 프로필을 단계 이름으로 나누면 필요한 부분만 띄워 볼 수 있다.

단계	Compose 프로필	올리는 컨테이너	완료 기준
M1	`ot`	hyd-sim, soft-plc, edgex-*, emqx-ot, fuxa	FUXA에 1 Hz 값이 보이고, FUXA 수동 조작으로 팬 속도가 바뀌며, 시뮬레이터 유온을 65 ℃ 넘기면 PLC가 트립한다.
M2	`backbone`	connect-ingest, kafka, connect-sink, timescaledb, grafana	Grafana에 `tag_1s` 추세가 보이고, `plant.wave`는 Kafka에만 쌓인다. IT를 내려도 FUXA는 계속 동작한다.
M3	`detect`	flink-jm, flink-tm (+ cmd-gateway 알람 중계)	시뮬레이터에 쿨러 열화를 주입하면 `alerts` RAISE → FUXA 알람, 복구하면 CLEAR.
M4	`monitor`	prom-agent, prometheus, alertmanager, mailpit	connect-ingest를 멈추면 수집 지연 경보 메일이 Mailpit에 도착한다.
M5	`knowledge`	neo4j	스키마·인스턴스 적재 후 T1 템플릿이 원인 후보 5개를, T2가 조치·SOP·매뉴얼 경로를 돌려준다.
M6	`agent`	mcp-kg, mcp-tsdb, mcp-prom, litellm, agent	`alerts` 한 건에서 근거가 인용된 가이드 카드 JSON이 생성되고, 인용이 빠지면 가드레일이 반려한다.
M7	`process`	bpmn, guide-app, cmd-gateway(명령)	승인 → `cmd/auto` → ACK DONE → 시뮬레이터 유온 하강 → 재관측 통과 → 종결. REMOTE_MANUAL 모드에서는 게이트웨이가 거부한다.

> **시뮬레이터 시간 가속.** 쿨러 열화(수십 분)와 15분 재관측을 프로토타입에서 기다리지 않도록 hyd-sim에 시간 배율(예: ×20)을 두고, BPMN 타이머와 CEP 시간창도 같은 배율 설정을 읽게 한다.

11.1 저장소 구조 제안

```
hyd-iot/
├ compose.yaml                # 프로필: ot, backbone, detect, monitor, knowledge, agent, process
├ .env                        # 비밀값, 시간 배율
├ ot/        hyd-sim/  soft-plc/  edgex/(device-daq, app-service 설정)  emqx/(acl.conf)  fuxa/(project)
├ dmz/       connect-ingest/  cmd-gateway/  prom-agent/
├ it/        kafka/  connect-sink/  flink/(jobs, models/*.onnx)  timescaledb/(init.sql)
│            grafana/(provisioning)  prometheus/(rules)  alertmanager/  neo4j/(schema.ttl, seed.cypher)
│            mcp/(kg, tsdb, prom)  agent/  litellm/  bpmn/  guide-app/
└ docs/      architecture-v3.pdf  topics.md  payload-schemas/*.json
```